Comparison between text-to-SQL methods

Simranjit Singh Dulku
Electrical and Computer Engineering
National Technical University of Athens
Team ID: 22
Student ID: 03121105

Fotis Monanteros
Electrical and Computer Engineering
National Technical University of Athens
Team ID: 22
Student ID: 03121912

Athanasios Markou
Electrical and Computer Engineering
National Technical University of Athens
Team ID: 22
Student ID: 03123618

***Abstract*—The translation of natural language questions into executable SQL queries, known as Text-to-SQL [1], [2], is a critical task for democratizing data access. While massive proprietary models have achieved state-of-the-art performance, their deployment is often limited by high computational costs and privacy concerns. This paper investigates the feasibility of using open-source Large Language Models (LLMs) with fewer than 2 billion parameters for this task in resource-constrained environments. We present a comparative analysis of two prominent LLMs, TinyLlama-1.1B and Qwen2.5-1.5B [3]–[6], evaluating their ability to generate valid SQL across varying levels of complexity. We developed a modular evaluation framework that integrates these models with two distinct Relational Database Management Systems, SQLite and PostgreSQL [7], [8], to assess dialect robustness. The system utilizes dynamic schema extraction and prompt engineering [9] techniques to mitigate the limited context windows inherent to smaller LLMs. Experiments were conducted using three standard datasets: Spider, ATIS, and Geography. Performance was measured using Execution Accuracy [10]–[12], verifying not just syntactic correctness but the validity of the data returned. Our results highlight the trade-offs between model size and reasoning capability, demonstrating that targeted optimization strategies can significantly enhance the utility of compact LLMs for lightweight database interfaces. To facilitate reproducibility, we have made our entire evaluation pipeline and datasets publicly available.[1]**

***Index Terms*—Text-to-SQL, Large Language Models, Qwen, TinyLlama, Execution Accuracy, Natural Language Processing, Database Systems**

I. Introduction

The rapid advancement of Large Language Models (LLMs) has revolutionized the interface between humans and structured data. The task of Text-to-SQL—automatically converting natural language questions into Structured Query Language (SQL)—has gained significant attention [1], [2] in the database community as a means to allow non-technical users to interact with Relational Database Management Systems (RDBMS). While massive proprietary models like GPT-4 [13] have demonstrated superior performance in this domain, they remain inaccessible for many applications due to data privacy regulations, latency requirements, and prohibitive inference costs.

This necessitates the exploration of more efficient, open-source LLMs that can be deployed locally or on edge devices [3], [4]. These compact LLMs offer the promise of reduced resource consumption and enhanced data privacy. However, they face significant challenges in the Text-to-SQL domain compared to their larger counterparts. Specifically, smaller LLMs often lack deep reasoning capabilities required for complex logic (e.g., nested subqueries or multi-table joins) and suffer from limited context windows, making it difficult to ingest large database schemas [1] without truncation.

In this work, we address these challenges by designing and implementing a robust Text-to-SQL agent capable of bridging the gap between lightweight LLMs and enterprise-grade databases. The primary objective is to evaluate and compare the performance of two distinct open-source LLMs: **TinyLlama-1.1B**, a compact generic chat model, and **Qwen2.5-1.5B**, a model optimized for instruction following and coding tasks.

Our contributions are as follows:

- **System Architecture [7], [8], [14]:** We developed a Python-based framework that integrates LLMs with two different RDBMS backends: **SQLite** for lightweight local testing and **PostgreSQL** to simulate a production environment. This dual-setup ensures the generated SQL is robust across different dialects.
- **Context Optimization [1]:** To overcome the limited context window of smaller LLMs (often 2048 tokens), we implemented dynamic schema extraction techniques. Instead of feeding raw DDL statements, our agent generates "dense" schema representations, reducing token usage by approximately 40% while retaining essential structural information.
- **Rigorous Evaluation [10]–[12]:** Moving beyond simple string matching, which often produces false negatives, we implemented an **Execution Accuracy** metric. The system executes both the generated query and the ground truth query on the live database and compares the resulting data sets, accounting for row and column ordering differences.
- **Complexity Analysis:** We utilized the **Spider** [2], **ATIS** [15], and **Geography** [16] datasets to test the models against a wide spectrum of query difficulties, from simple `SELECT-WHERE` clauses to complex joins and aggregations.

The remainder of this paper details the methodology used to construct the agent, the specific prompt engineering strategies employed to stabilize LLM output, and a quantitative analysis of the results. By systematically benchmarking these models,

[1]Code and Data: https://github.com/Fotismon/text2sql-project

we aim to provide a clear perspective on the viability of compact LLMs for next-generation database interfaces.

II. Infrastructure and Software

To ensure reproducibility, resource efficiency, and portability, the experimental framework was developed using an open-source modular software stack and executed on cloud infrastructure. The system architecture was designed to mitigate the resource constraints inherent in the Google Colab Free Tier while maintaining rigorous evaluation standards.

A. Hardware Environment

All experiments were conducted on the Google Colab [17] platform, utilizing a high-performance compute instance provided via the Free Tier allocation. The hardware specifications were as follows:

- **GPU Acceleration:** NVIDIA T4 [18] Tensor Core with 16 GB GDDR6 VRAM. This accelerator provides sufficient memory to load Large Language Models (LLMs) in the 1–2 billion parameter range without aggressive quantization. Both 1.1B and 1.5B parameter models were loaded in 16-bit half-precision floating-point format (FP16), occupying approximately 3–4 GB of VRAM each. This left adequate memory overhead for the PyTorch [19] runtime and the active context window.
- **Central Processing:** Intel Xeon CPUs (2 vCPUs @ 2.20 GHz), responsible for data preprocessing, orchestration logic, and database interaction overhead.
- **System Memory:** 12.7 GB RAM. This memory capacity was critical for buffering dataset replicas during the extract–transform–load (ETL) process between heterogeneous database engines.

B. Software Stack

The system was implemented in Python 3.10 with strictly version-controlled dependencies to ensure deterministic execution. The architecture was built upon three primary software components:

1) Deep Learning and Inference Engine: Model orchestration was implemented using the Hugging Face `Transformers` [9] library. To optimize execution on the NVIDIA T4 GPU, the `Accelerate` [20] library was used with the `device_map="auto"` configuration, enabling automatic placement of tensor operations across available hardware resources.

Importantly, no 4-bit or 8-bit quantization techniques were applied. All models were executed with native FP16 precision, preserving the full reasoning capacity of LLMs and eliminating quantization-induced error as a confounding factor in performance evaluation.

2) Database Interaction Layer: The database backend relied on a robust, cross-dialect interaction layer to support multiple relational engines:

- **SQLAlchemy [14]:** Served as the Object–Relational Mapper (ORM) and abstraction layer, enabling seamless switching between SQLite and PostgreSQL backends without modifying core query logic.
- **psycopg2-binary [21]:** Used as the PostgreSQL adapter, compliant with the Python Database API Specification v2.0 and optimized for high-performance query execution.
- **SQLParse [22]:** Used for static SQL analysis to calculate query complexity (e.g., counting joins and nested subqueries) and to normalize SQL syntax before comparison.

3) Data Management and ETL: Data preprocessing and migration were implemented using the `Pandas` library as part of a custom ETL pipeline. A critical component of this pipeline was **cross-system synchronization**. By leveraging SQLAlchemy's abstraction layer [14], we automated the extraction of table schemas and data records from the source repositories and their subsequent loading into the target RDBMS [14]. This automated approach ensured that the **PostgreSQL** and **SQLite** [7], [8] environments contained identical dataset states prior to evaluation, minimizing structural discrepancies that could confound the experimental results.

C. Database Systems

To evaluate query portability and robustness, a dual-engine database environment was employed:

- **SQLite (v3.x):** Used as an embedded database for rapid schema introspection and integration with Spider dataset artifacts. Its file-based architecture enabled lightweight experimentation and fast iteration [7].
- **PostgreSQL (v15):** Deployed as a background service within the execution runtime and treated as the production-grade reference system. Executing model-generated queries against PostgreSQL ensured compliance with strict ANSI SQL semantics and prevented reliance on SQLite's permissive typing and execution behavior [8].

D. Development Lifecycle and Reproducibility

A hybrid development workflow was adopted to ensure code robustness and experimental validity:

1) Local Development Phase: Core agent logic and database interaction modules were initially developed locally using Visual Studio Code (VSCode). A Dockerized PostgreSQL container was used to simulate the production environment, allowing rapid debugging, schema validation, and isolated development of the ETL pipeline in a fully reproducible setting [23].

2) Cloud Execution Phase: The final experimental benchmarks were executed on Google Colab. Since Docker [23] is not natively supported in the standard Colab runtime, the environment was provisioned via an automated setup script that installed the exact PostgreSQL version and Python dependencies used during local development. This ensured behavioral consistency between local and cloud executions, preserving the integrity of the experimental results.

III. System Implementation and Methodology

This section describes the system-level methodology used to evaluate compact open-source LLMs for Text-to-SQL. While Section II summarizes the execution environment and software stack, we focus here on the pipeline design, the prompt construction procedure, and the evaluation protocol that ensures fair and repeatable benchmarking.

The framework is implemented as a modular pipeline with explicit separation between model inference, database interaction, and evaluation. This separation enables controlled experiments where the model, dataset, or backend can be changed independently, without altering the remainder of the workflow. Each run therefore follows a fixed sequence of steps: schema acquisition, prompt construction, SQL generation, query execution, and result-based scoring.

A key methodological challenge for small models is the limited context window, which makes it impractical to include full database definitions in every prompt. To address this, the system performs runtime schema introspection and converts the schema into a compact representation that preserves the information required for query synthesis, such as table names, column names, and essential relational structure. The prompt is then composed by combining the natural language question with this condensed schema description and a strict formatting constraint that encourages the model to output a single executable SQL statement.

Model integration is implemented through a unified inference interface. Each model is wrapped to expose a common API for generation, ensuring consistent decoding settings and comparable behavior across experiments. Lightweight post-processing is applied to stabilize the output format and reduce superficial variability, without changing SQL semantics. This allows the evaluation to reflect differences in reasoning and SQL formulation rather than differences in formatting.

Evaluation is performed using execution-based comparison rather than string matching [10]–[12]. For each input question, both the predicted SQL query and the corresponding ground-truth query are executed on the selected database backend under identical constraints. The resulting outputs are normalized to reduce false mismatches due to nondeterministic row ordering or representation differences. A prediction is considered correct only when the normalized result set matches the ground-truth result set. This evaluation criterion is reported as Execution Accuracy and directly reflects semantic correctness, since multiple syntactically different queries may still be equivalent.

During each run, the framework logs all relevant artifacts, including the input question, the generated SQL, execution status, runtime statistics, and the final correctness label. This structured logging supports subsequent analysis of error modes (e.g., missing joins, incorrect aggregation, or invalid column references) and enables reproducible reporting across datasets and model configurations.

A. Data Preparation and Mirroring

A core requirement of our experimental design was to ensure *data consistency* between the two RDBMS backends, so that any performance differences could be attributed to the generated SQL (and potential dialect effects), rather than to discrepancies in the underlying database contents. To achieve this, we adopted a two-stage data preparation workflow: (i) acquiring the datasets in a reproducible, structured form, and (ii) mirroring the same database state across SQLite and PostgreSQL.

a) Dataset acquisition and organization.: We relied on three established Text-to-SQL benchmarks—Spider, ATIS, and Geography—covering diverse schemas and a broad range of query complexity. For Spider, we used the official distribution [2], [11], which provides complete database files alongside the corresponding schema definitions and question–SQL pairs. For ATIS [15] and Geography [16], we used a lightweight JSON-based distribution that includes the natural language questions and ground-truth SQL queries. All dataset assets were placed under a dedicated `data/` directory, following a consistent layout that separates database files (used for execution) from the question sets (used for evaluation). This organization ensures that dataset loading remains deterministic and easily reproducible across machines and runs.

b) Database mirroring across engines.: To guarantee identical data content across SQLite and PostgreSQL, we implemented a dedicated loader utility (`db/load_dataset.py`). The loader automatically locates the source SQLite database files corresponding to a selected dataset/database identifier, enumerates all tables in the source database, and transfers their contents into the target backend. The mirroring is performed at the table level: each table is extracted from the source database and then recreated in the target database with its full row content, ensuring that subsequent executions of both predicted and ground-truth queries are performed over equivalent data.

This mirroring step is critical for our execution-based evaluation protocol. Since our primary metric is Execution Accuracy, correctness is determined by comparing the result sets of the predicted query against the result sets of the ground-truth query. Such a comparison is only meaningful if both queries operate on the same data distribution and database state. By replicating the data from the canonical source database into PostgreSQL, we ensure that query execution remains comparable across backends, while also enabling robustness checks for dialect-dependent behavior.

c) Consistency checks and reproducible resets.: The loader was designed to support repeated experimental runs without manual database cleanup. Before inserting new data, tables are recreated in a controlled manner, preventing residual state from previous runs. In addition, the successful completion of the data preparation stage is verified through basic integrity checks, such as confirming that the expected tables exist in both engines and that sample queries return plausible results. This practice reduces the likelihood of silent data load-

ing failures and strengthens the reliability of the experimental results.

d) Unified question format for downstream evaluation.: In parallel with database preparation, the question sets from the different datasets were converted into a unified internal representation. Although Spider, ATIS, and Geography distribute their samples using slightly different JSON schemas, all samples were normalized to a common format that includes (at minimum) the natural language question, the ground-truth SQL query, a database identifier (when applicable), and a coarse query complexity label. This normalization enables the evaluation pipeline to iterate over all datasets uniformly and supports stratified reporting of model performance by difficulty.

e) Dataset statistics.: The final distribution of the prepared data, stratified by complexity, is presented in Table I. Notably, the *Spider* dataset provides the highest density of "Hard" and "Extra Hard" queries, serving as the primary benchmark for complex reasoning in our experiments.

TABLE I
DISTRIBUTION OF QUERY COMPLEXITY BY DATASET

Dataset	Easy	Medium	Hard	Extra Hard	Total
ATIS	268	420	3,835	757	5,280
Geography	484	32	1	360	877
Spider	3,466	4,046	670	1,511	9,693
Custom	25	25	25	25	100
Total	**4,243**	**4,523**	**4,531**	**2,653**	**15,950**

B. Model Integration (Model Wrappers)

To standardize interactions with different Large Language Models (LLMs) and ensure fair comparisons, we implemented a lightweight abstraction layer in `agent/model_wrappers.py`. At its core, the framework defines a common interface through an abstract `LLMWrapper` class, which encapsulates shared responsibilities such as device selection (CPU/GPU), tokenizer/model initialization, and a unified method for SQL generation. Model-specific subclasses then implement only the components that differ across architectures, allowing the rest of the pipeline (prompting, execution, and evaluation) to remain model-agnostic.

Two wrappers were implemented in this work:

- **TinyLlamaWrapper:** `TinyLlama-1.1B-Chat-v1.0` [3], [5]
- **Qwen2Wrapper:** `Qwen2.5-1.5B-Instruct` [4], [6]

Both models are loaded through the `Hugging Face Transformers` stack [9] and executed in half precision (`torch_dtype=float16`) to reduce memory footprint and enable inference on a single commodity GPU. In addition, `trust_remote_code=True` was enabled to guarantee compatibility with model repositories that define custom generation logic or specialized tokenization behavior. This was particularly important for instruction-tuned models where the recommended inference path may rely on repository-provided implementations.

Crucially, the wrappers abstract away the divergence in prompting requirements; the Qwen wrapper utilizes the model's native chat template (`apply_chat_template`) [9], while the TinyLlama wrapper implements a raw completion prompt optimized for its specific training data.

A key integration challenge in Text-to-SQL is ensuring that the model output is directly executable. Even when prompted to return only SQL, instruction-tuned chat models frequently include conversational markers, role tokens, or Markdown code fences. To address this, both wrappers include a dedicated post-processing stage implemented through `_clean_sql`. This function applies deterministic normalization rules, primarily using regular expressions, to remove common non-SQL artifacts (e.g., Markdown delimiters such as ``` ``` style fences, inline annotations, or chat-role fragments) and to extract the first complete SQL statement. The output of the wrapper is therefore a single, plain SQL string, suitable for immediate execution by the downstream database layer.

Finally, to validate correct installation and integration of the wrapper layer before running large-scale experiments, we employed a minimal functional test routine. This routine instantiates each wrapper, submits a simple prompt containing a toy schema and a short natural language query, and verifies that the returned output conforms to the expected "single SQL statement" constraint. This step reduces debugging time by isolating model-loading and output-formatting issues prior to full dataset evaluation.

C. Context Optimization and Schema Truncation

A practical limitation of compact LLMs is their restricted context window (approximately 2048 tokens), which is frequently exceeded by real-world database schemas when represented in full Data Definition Language (DDL) form [1]. To address this constraint, we implemented a schema compaction strategy that produces a prompt-friendly representation while preserving the structural cues required for SQL generation.

Specifically, the framework constructs a *semi-compact* schema string via a dedicated routine in `utils/sql_utils.py`. The method first resolves the schema source from the dataset directory (prioritizing explicit `schema.sql` files when available) and then removes non-essential content such as comments, data manipulation statements, and auxiliary objects (e.g., indexes, triggers, and views). For each `CREATE TABLE` block, the representation retains the table name and a minimal list of column identifiers, while preserving `FOREIGN KEY`/`REFERENCES` declarations to maintain join-relevant information. This design yields a substantially shorter and less noisy schema description, allowing large databases to be included in the prompt without exceeding the model's input limit.

When an explicit schema file is not available, the framework falls back to deriving schema information directly from the underlying SQLite metadata (using `sqlite_master`) [7]. In this case, table definitions are extracted in a deterministic order, supporting stable and repeatable model behavior across

runs. In addition, schema strings are cached per database identifier during the experiment loop, ensuring that truncation and compaction are performed once per database and reused for all associated queries, thus reducing overhead and improving throughput.

D. Prompt Engineering and Controlled Decoding

To ensure consistent interaction across heterogeneous LLMs, we adopted a strict prompting protocol implemented in `agent/model_wrappers.py`. Each model wrapper enforces a constrained generation setting designed for executable output: the prompt explicitly instructs the model to use only the provided schema elements and to return only a valid SQL statement, without explanation or formatting.

Given differences in instruction-tuning conventions, the system uses model-specific input formatting. For Qwen2.5, inputs are constructed using role-based messages and serialized through the model's chat template, whereas TinyLlama is queried using a completion-style prompt that empirically yields more stable SQL-only outputs under the same constraints. In both cases, inputs are tokenized with truncation enabled and capped at the model's context length, ensuring that the most relevant portion of the schema and the question are retained.

To reduce output variance and prevent conversational continuation beyond the SQL query, generation is performed with greedy decoding (sampling disabled) and a bounded output budget (a small maximum number of newly generated tokens, set conservatively per model) [9]. Finally, the produced text is passed through a post-processing stage (`_clean_sql`) that removes common artifacts such as Markdown code fences or residual assistant tokens and, when applicable, truncates the output to a single SQL statement. This normalization step ensures that downstream execution and evaluation operate on clean, database-executable queries.

In addition to the base prompt, the evaluation loop includes a lightweight, execution-driven correction mechanism: if the initially generated SQL fails to execute on SQLite (excluding timeouts and resource-related failures), a single retry is triggered by augmenting the original user query with the database error message and an explicit instruction to produce a corrected query using only the schema [10]. This controlled retry improves robustness against minor syntactic and dialect-related mistakes without introducing multi-turn interaction complexity.

E. Experiment Loop

Building upon the context optimization and prompting strategy described in the previous subsections, the experiment loop consumes the cached semi-compact schema representation directly during prompt construction and enforces strict execution timeouts to ensure that results are comparable across models and backends.

The end-to-end evaluation procedure is implemented in `experiments/run_experiment.py`, which orchestrates model inference, database preparation, query execution, and result logging in a single reproducible pipeline. The script is designed for execution in resource-constrained settings (e.g., a single NVIDIA T4 GPU) and follows a strictly controlled workflow that prioritizes robustness over maximum throughput.

a) Single-model execution and latency tracking.: To avoid GPU memory fragmentation and reduce variability, each run evaluates a single LLM instance selected via a command-line argument. The script strictly isolates the inference step to measure **generation latency** (in seconds) for every query. This run-level efficiency metric is recorded alongside correctness labels, enabling a direct trade-off comparison between model reasoning time and accuracy.

b) Dataset iteration and optimized grouping.: For each selected dataset, the pipeline loads the available questions and applies sampling to cap the number of evaluated instances per dataset if configured. Crucially, the question list is sorted by `db_id` prior to execution. This sorting ensures that queries referring to the same database are processed in contiguous blocks, minimizing context switching overhead.

c) Batch database preparation and schema caching.: To minimize runtime latency during inference, the system employs a batch preparation strategy. Before entering the generation loop, the script identifies all unique database identifiers (`db_id`) required for the current sample. It then iterates through this unique set to: (1) trigger the dataset loader to mirror the specific SQLite databases into PostgreSQL, and (2) construct and cache the filtered schema strings in memory. This ensures that all necessary database contexts are available in the cache prior to model execution, eliminating repeated I/O operations for questions sharing the same underlying database.

d) Execution-based evaluation and error correction.: For each question, the ground-truth SQL query is executed on SQLite under a strict timeout to obtain the reference result set [24]. The model then generates a candidate SQL query conditioned on the question and the cached schema string. The generated query is executed on SQLite under the same timeout constraint. To improve system robustness, the pipeline employs a **reflexive correction mechanism**: if the initial query fails due to a syntax or execution error (excluding timeouts), the system performs a single retry by feeding the error message back to the model [10]. Correctness is determined by a set-based comparison of the final result set against the ground-truth results [12].

e) Resumable logging.: All per-question outputs are logged to a CSV file, including the natural language input, ground-truth SQL, generated SQL, execution success flags, correctness label, and generation latency. To support long-running experiments and prevent data loss, results are flushed to disk periodically. The script also includes logic to detect previously completed questions based on their unique identifiers, allowing interrupted runs to resume execution deterministically without re-processing completed samples.

F. Colab Execution Workflow

All experiments were executed on Google Colab [17] to leverage a cloud GPU environment while maintaining a fully reproducible, script-driven workflow. The notebook is structured into sequential cells that prepare storage, fetch the repository, configure the database backends, populate datasets, and finally run the experiment loop and analysis.

a) Drive integration and workspace initialization.: At the beginning of each session, Google Drive is mounted to provide persistent storage for intermediate artifacts and final outputs. A dedicated results directory is created inside Drive and is later passed to the experiment runner, ensuring that logs and CSV outputs survive Colab runtime resets. The project repository is then cloned into the Colab filesystem. For private repositories, authentication is performed securely by requesting a GitHub Personal Access Token through a non-echoed input prompt, avoiding accidental exposure in notebook outputs. To prevent duplicating large dataset files, the notebook links the dataset directory stored in Drive into the project root (Drive-to-Colab linkage), so the code can access the data under a consistent local path.

b) Dependency installation.: After the repository is available, the Python environment is prepared by installing the pinned project dependencies. In addition to the core database and utility libraries, the notebook installs the required model inference stack (Transformers, PyTorch, Accelerate) and auxiliary packages used by the pipeline. This ensures that both model wrappers and execution utilities behave consistently across runs.

c) PostgreSQL setup within Colab [8].: PostgreSQL is installed directly in the Colab VM using the system package manager. The service is started within the runtime and configured with a dedicated database and user whose credentials match those expected by the project's database connection layer [8]. This step establishes a production-like RDBMS backend that can be used alongside SQLite to evaluate cross-engine executability.

d) Database population and consistency verification.: Once the backends are available, the notebook initializes the local SQLite database file and invokes the dataset loader to populate both SQLite and PostgreSQL with the selected datasets. A verification step is executed afterwards to confirm that the mirroring process has completed successfully and that the two backends contain consistent content, which is necessary for meaningful execution-based evaluation.

e) Custom dataset construction.: Beyond the standard benchmarks, the notebook includes a dedicated section for creating a small custom dataset. A schema file is defined for the custom database, after which auxiliary scripts generate the corresponding database instance and produce synthetic natural-language questions paired with ground-truth SQL queries. This dataset is stored alongside the benchmark datasets and becomes available to the same unified loader and experiment runner.

f) Running experiments and producing SQL.: The main evaluation is executed by calling the experiment runner script with explicit command-line parameters. Experiments are launched separately per model (e.g., Qwen2.5-1.5B and TinyLlama-1.1B), while keeping the dataset list and evaluation configuration fixed. The notebook specifies:

(i) the datasets to include (Spider, ATIS, Geography, and the custom set),
(ii) the number of questions per dataset,
(iii) strict execution timeouts for SQLite and PostgreSQL, and
(iv) a flush frequency that forces frequent writes to the results CSV stored in Drive.

Between model runs, memory cleanup commands are executed to reduce the risk of GPU memory fragmentation and to stabilize subsequent runs.

g) Results analysis.: Finally, a post-processing script is executed to aggregate the produced CSV logs and compute summary statistics for reporting. This analysis step generates the figures and tables used in the experimental section, enabling a consistent transformation from raw execution logs to the final metrics presented in the paper.

IV. Results and Analysis

In this section, we present a comprehensive evaluation of the two Large Language Models (LLMs), **Qwen2.5-1.5B** and **TinyLlama-1.1B**. The analysis focuses on three key dimensions: quantitative accuracy across datasets, reasoning capability under varying complexity, and operational efficiency (latency and robustness).

A. Overall Performance Overview

We first evaluate the aggregate performance across all tested datasets. Table II summarizes the key metrics. **Qwen2.5-1.5B** demonstrated a significant performance advantage, achieving an overall accuracy of **28.45%** compared to TinyLlama's **8.60%**—a nearly $3.3\times$ improvement.

TABLE II
Overall Model Performance Summary

Model	Total Samples	Accuracy (%)	Exec. Rate (%)
Qwen2.5-1.5B	15,950	**28.45**	**57.05**
TinyLlama-1.1B	15,950	8.60	31.12

Figure 1 visualizes the critical trade-off between strict accuracy and execution stability. Qwen's execution success rate (57.05%) was nearly double that of TinyLlama. This indicates that Qwen has a much stronger grasp of valid SQL syntax and database schema constraints, whereas TinyLlama frequently generates hallucinated column names or syntactically invalid SQL (e.g., missing `GROUP BY` clauses).

B. Dataset Performance and Complexity Analysis

To understand the factors driving performance, we analyze the execution accuracy across varying dataset domains and query complexities. Figure 2 and Table III present the high-level performance breakdown by dataset.

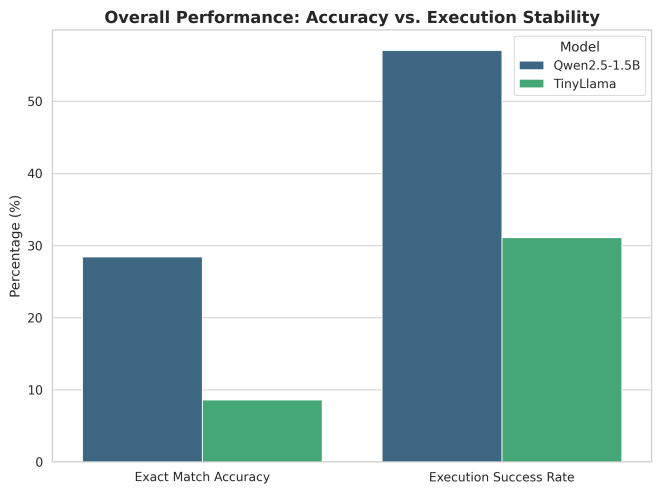

Fig. 1. **Overall Performance.** Qwen2.5 (blue) dominates in both Exact Match Accuracy and Execution Success Rate, highlighting its superior instruction-following capabilities compared to TinyLlama (green).

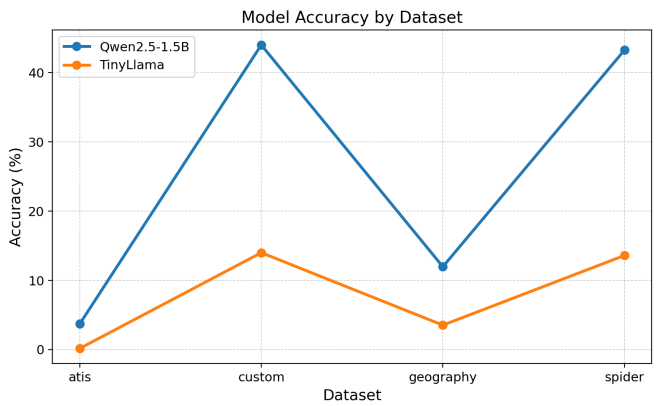

Fig. 2. **Accuracy by Dataset.** Qwen2.5 achieves respectable performance on Spider and Custom datasets (general reasoning) but struggles significantly with the domain-specific schemas of ATIS and Geography.

TABLE III
EXECUTION ACCURACY (%) ACROSS DATASETS

Model	Dataset	Accuracy (%)
Qwen2.5-1.5B	ATIS	3.71
	Custom	44.00
	Geography	11.97
	Spider	43.26
TinyLlama-1.1B	ATIS	0.17
	Custom	14.00
	Geography	3.53
	Spider	13.60

The results indicate a clear dichotomy:

- **General Reasoning (Spider & Custom):** On benchmarks testing logical generalization across standard schemas, Qwen2.5 achieved a robust **43.26%**, tripling TinyLlama's score. This confirms that 1.5B parameter models can handle standard SQL logic (Joins, Aggregates) when the column names are semantically clear.
- **Schema Linking Failure (ATIS & Geography):** Both models collapsed on *ATIS* ($< 4\%$) and *Geography* ($< 12\%$). These datasets rely on abbreviated column codes and domain-specific knowledge, confirming that without fine-tuning, LLMs cannot bridge the semantic gap regardless of their SQL syntax knowledge.

1) Spider Dataset (General Reasoning): Spider serves as our primary benchmark as it is considered the best overall Text-to-SQL dataset. Figure 3 visualizes the performance degradation as complexity increases.

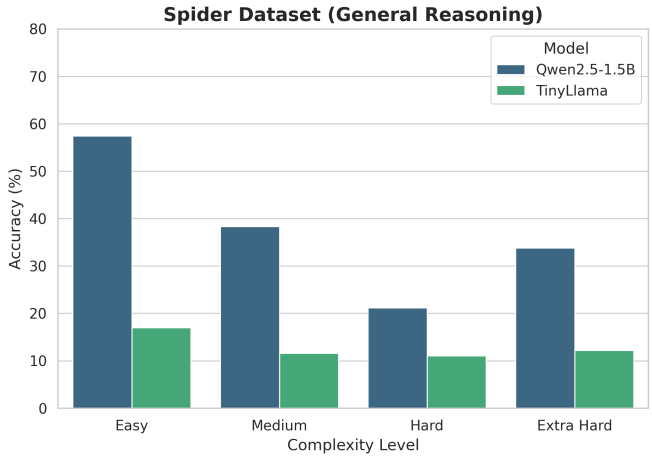

Fig. 3. **Spider Accuracy by Complexity.** Qwen2.5 exhibits a standard reasoning curve but shows surprising resilience on "Extra Hard" queries. TinyLlama flatlines, unable to handle schema linking even on simple tasks.

Table IV details the numerical breakdown. Qwen2.5 exhibits a "reasoning curve" typical of larger models: it performs well on *Easy* queries (57.4%) but drops on *Hard* queries. Notably, Qwen displays resilience in the *Extra Hard* category (33.82%), outperforming its own score on *Hard* queries. Qualitative analysis suggests the model handles nested subqueries (common in "Extra Hard") better than specific set operations like `EXCEPT` or `INTERSECT` (common in "Hard").

In contrast, TinyLlama's performance is uniformly low ($\sim$11-17%), indicating a fundamental bottleneck in **Schema Linking** rather than logical reasoning.

TABLE IV
ACCURACY BREAKDOWN BY COMPLEXITY (SPIDER DATASET)

Model	Easy	Medium	Hard	Extra Hard
Qwen2.5-1.5B	**57.41%**	**38.31%**	**21.19%**	**33.82%**
TinyLlama-1.1B	17.02%	11.62%	11.04%	12.18%

2) ATIS Dataset (Domain Specific): The ATIS dataset (Figure 4) presents a "Schema Linking" challenge. Even on "Easy" queries, both models fail significantly ($< 12\%$ accuracy). This confirms that without fine-tuning, the models cannot effectively bridge the gap between natural language terms (e.g., "New York") and abbreviated database codes (e.g., "NYC").

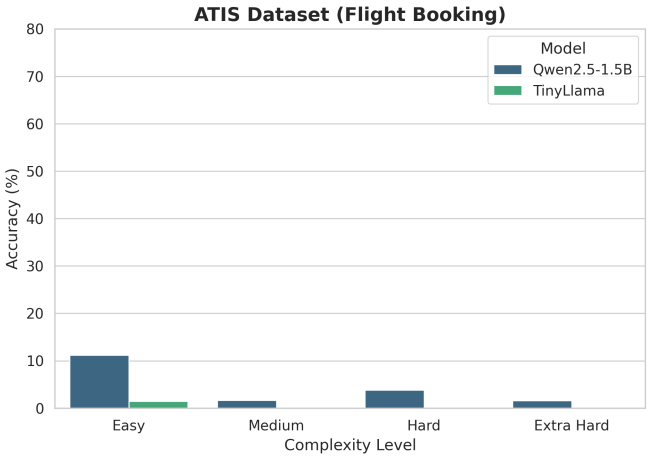

Fig. 4. **ATIS Accuracy by Complexity.** Both models fail across all complexity levels, confirming the limitations of zero-shot prompting on domain-specific schemas.

3) Custom Dataset: The Custom dataset (Figure 5) clearly illustrates the "Complexity Wall" for TinyLlama. It performs decently on Easy/Medium tasks (24-32%) but drops to exactly 0% on Hard/Extra Hard tasks, confirming it lacks the capacity for nested logic.

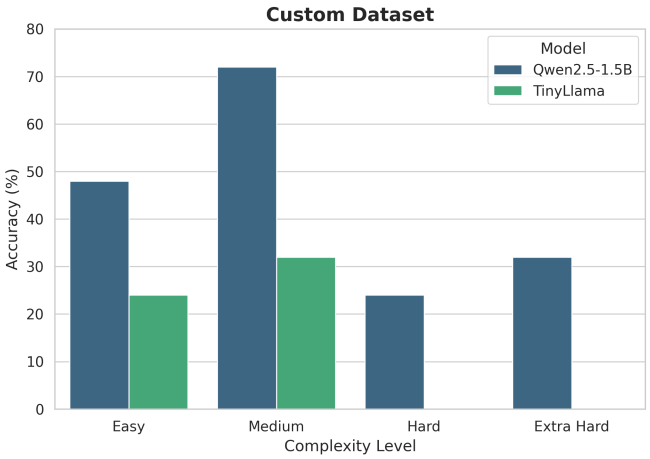

Fig. 5. **Custom Dataset Accuracy.** TinyLlama hits a "Complexity Wall" on Hard queries (0% accuracy), while Qwen2.5 retains reasoning capabilities across all levels.

4) Geography Dataset: Similar to ATIS, the Geography dataset challenges the models with domain-specific entities. As shown in Figure 6, performance is low across the board. The failure here is primarily attributed to the schema-linking problem, where models fail to map natural language expressions (e.g., "bordering states") to specific encoded column values.

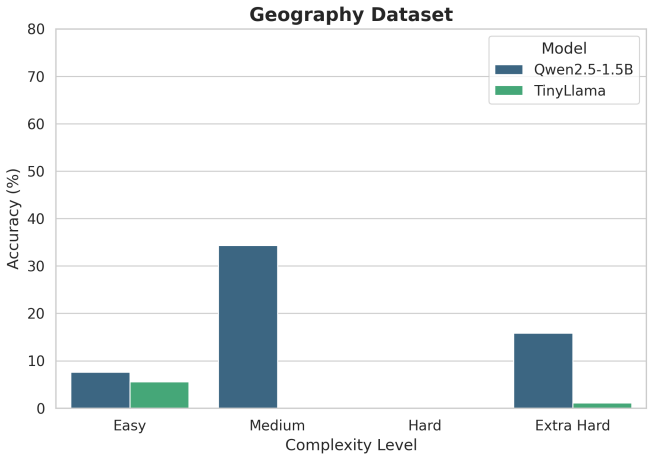

Fig. 6. **Geography Dataset Accuracy.** Performance is severely limited by the lack of semantic grounding in the zero-shot prompt, though Qwen2.5 still manages to solve 35% of Medium queries.

C. Operational Efficiency

For deployment on edge devices, inference latency is a critical metric. We analyze this from two perspectives: per-query latency, total runtime and resource profiling.

1) Average Latency: Figure 7 presents the average generation time per query. Counter-intuitively, the larger model (Qwen2.5-1.5B) is consistently faster than the smaller model (TinyLlama-1.1B).

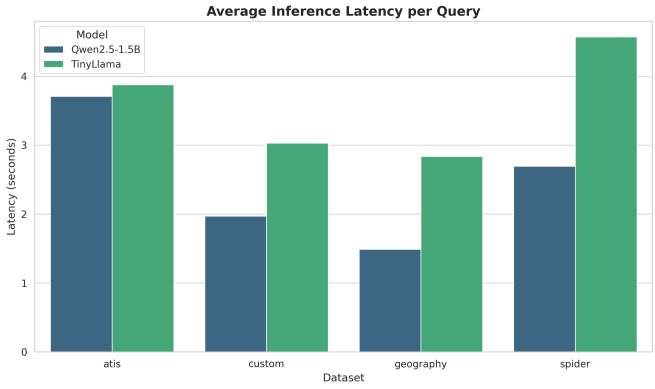

Fig. 7. **Inference Latency per Query.** Despite having more parameters, Qwen2.5 is faster. This is driven by its ability to generate concise, valid SQL without the conversational filler that plagues TinyLlama.

On Spider, Qwen is approximately **41% faster**. This efficiency gap is driven by output quality: TinyLlama frequently fails to adhere to the system prompt, generating long chains of "conversational filler" or repetitive invalid tokens until the hard token limit is reached. Qwen2.5, being instruction-tuned, typically outputs a concise SQL query and triggers the End-of-Sequence (EOS) token immediately.

TABLE V
AVERAGE INFERENCE LATENCY (SECONDS PER QUERY)

Model	Spider	ATIS	Geography	Custom
Qwen2.5-1.5B	**2.69**	**3.71**	**1.49**	**1.97**
TinyLlama-1.1B	4.57	3.88	2.84	3.03

2) Total Runtime: When aggregated over the full benchmark suite (Figure 8), the latency difference accumulates into an operationally significant gap. Qwen2.5 completed the full evaluation suite in **786 minutes**, reducing the total execution time by **339 minutes (30%)** compared to TinyLlama's 1,125 minutes. This substantial reduction in GPU-hour consumption makes Qwen2.5 both more accurate and more cost-effective for batch processing.

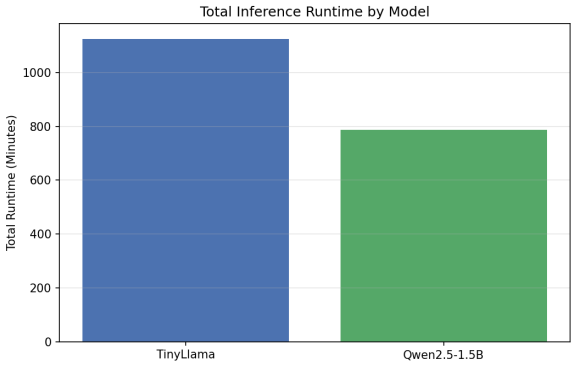

Fig. 8. **Total Inference Runtime.** The cumulative effect of TinyLlama's verbose outputs results in a much longer total runtime (blue), whereas Qwen2.5 (green) is highly efficient.

3) Resource Profiling: Resource profiling was conducted on a stratified subsample of 400 queries (100 per dataset) to isolate hardware performance metrics. We tracked peak GPU VRAM consumption and the System RAM footprint during active inference.

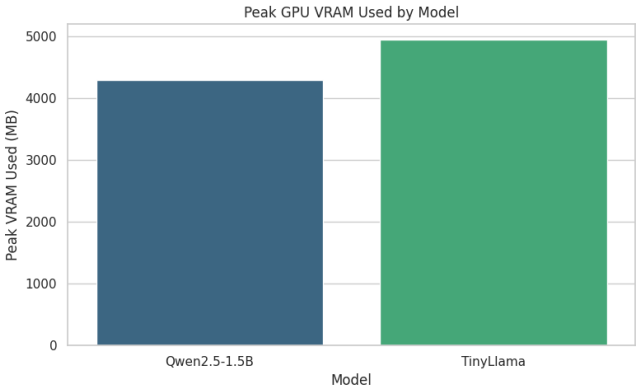

Fig. 9. Peak VRAM Usage.

a) The Efficiency Paradox: A counter-intuitive finding of this study is the *Efficiency Inversion*: the larger model

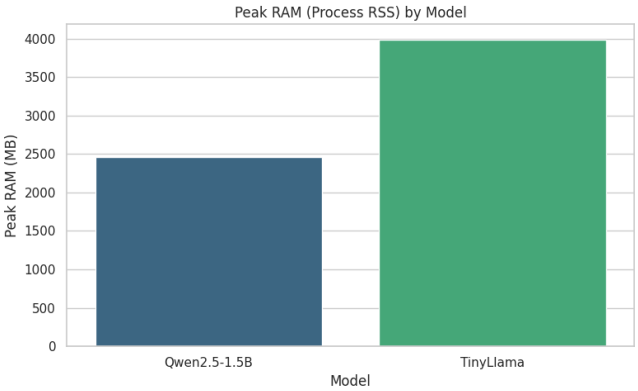

Fig. 10. Peak System RAM Usage.

(Qwen2.5-1.5B) proved significantly more efficient than the smaller model (TinyLlama-1.1B) across all hardware dimensions.

- **Memory Footprint:** Despite its larger parameter count, Qwen consumed **13% less GPU VRAM** (4.3GB vs. 4.9GB) and **38% less System RAM** (2.5GB vs. 4.0GB).

This inversion is driven by **Output Conciseness**. TinyLlama frequently failed to adhere to the system prompt ("Output ONLY SQL"), generating long chains of conversational filler or invalid tokens until hitting the hard context limit. This verbose generation unnecessarily bloated the Key-Value (KV) cache, directly increasing VRAM usage and processing time. In contrast, Qwen2.5's robust instruction tuning enabled it to generate precise SQL queries and trigger the End-of-Sequence (EOS) token immediately, minimizing computational overhead.

b) Edge Deployment Viability: With a peak VRAM usage of just 4.3GB and a system RAM footprint of 2.4GB, **Qwen2.5-1.5B** fits comfortably within the constraints of modern edge accelerators, such as the NVIDIA Jetson Orin Nano or consumer-grade GPUs with 8GB VRAM. This efficiency leaves ample memory headroom for the application layer and database engine, which is critical for local-only deployments.

D. Dialect Robustness

We measured *Postgres Compatibility*: the percentage of queries valid in SQLite that also executed successfully in the stricter PostgreSQL environment.

As shown in Figure 11, Qwen2.5 achieves a higher compatibility score (31.6%) compared to TinyLlama (24.2%). TinyLlama relies more heavily on SQLite's lenient typing system (e.g., implicit casting). When moved to PostgreSQL, its queries frequently fail due to type mismatches and syntax errors, highlighting the importance of using models with strong code-specific pre-training for production environments.

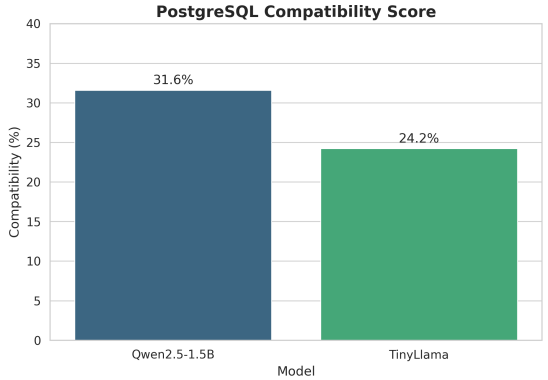

Fig. 11. **Dialect Robustness.** Qwen2.5 achieves higher compatibility (31.6%), indicating it produces standard ANSI SQL. TinyLlama relies on SQLite-specific leniencies.

V. Conclusion and Future Work

This study presented a rigorous evaluation of Large Language Models (LLMs) for the Text-to-SQL task, focusing on their viability in resource-constrained environments. By architecting a dual-engine evaluation framework (PostgreSQL and SQLite) and implementing dynamic schema optimization techniques, we were able to benchmark **TinyLlama-1.1B** and **Qwen2.5-1.5B** across diverse query complexities.

Our findings lead to four critical conclusions regarding the deployment of LLMs:

1) **The Efficiency Paradox:** Contrary to the standard assumption that smaller models are faster, we observed that the 1.5B parameter model (Qwen2.5) was approximately **40% faster** than the 1.1B model (TinyLlama). This inversion is driven by instruction adherence: the instruction-tuned Qwen model generated concise, valid SQL and terminated immediately, whereas the generic TinyLlama model wasted computational resources generating conversational artifacts and invalid tokens. This suggests that for code generation tasks, *quality of instruction tuning* is a more significant predictor of latency than raw parameter count.
2) **Reasoning vs. Recall:** While Qwen2.5 demonstrated surprising resilience on "Extra Hard" queries (33.8% accuracy), effectively handling nested subqueries, both models collapsed on domain-specific datasets like ATIS ($< 4\%$ accuracy). This delineates a clear boundary for zero-shot LLMs: they possess the *logical reasoning* to construct complex SQL syntax but lack the *semantic knowledge* to perform zero-shot schema linking on abbreviated or obscure database schemas.
3) **The "Complexity Wall":** Our results identified a sharp "Complexity Wall" for generic LLMs. TinyLlama's accuracy dropped to exactly 0% on hard queries in the Custom dataset, indicating that below a certain threshold of model capacity (or specific training), the ability to maintain context for nested logic completely evaporates.
4) **Production Readiness:** The dialect robustness analysis revealed that relying on SQLite-based evaluation overestimates model capability. Qwen2.5 demonstrated a 31.6% Postgres compatibility score, producing standard ANSI SQL, whereas TinyLlama relied heavily on SQLite's permissive typing. For production systems, this underscores the necessity of evaluating models against strict-dialect engines to prevent runtime failures in enterprise environments.

Future Work: Given the observed bottleneck in schema linking, future research should prioritize **Retrieval-Augmented Generation (RAG)** techniques to inject semantic column descriptions into the prompt dynamically. Additionally, exploring parameter-efficient fine-tuning (e.g., LoRA) on domain-specific corpora could potentially bridge the gap observed in the ATIS and Geography datasets, allowing LLMs to combine their syntactic reasoning with domain-specific vocabulary.

References

[1] N. Deng, Y. Chen, and Y. Zhang, "Recent advances in text-to-sql: A survey of what we have and what we expect," in *Proceedings of COLING*, 2022.
[2] T. Yu *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," in *Proceedings of EMNLP*, 2018.
[3] P. Zhang *et al.*, "Tinyllama: An open-source small language model," arXiv preprint, 2024.
[4] Qwen Team, "Qwen2.5 technical report," Technical report / arXiv preprint, 2024.
[5] TinyLlama Team, "Tinyllama/tinyllama-1.1b-chat-v1.0 (model card)," Hugging Face model card, 2024.
[6] Qwen Team, "Qwen/qwen2.5-1.5b-instruct (model card)," Hugging Face model card, 2024.
[7] SQLite Consortium, "Sqlite documentation," Official documentation, 2024.
[8] PostgreSQL Global Development Group, "Postgresql documentation," Official documentation, 2024.
[9] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of EMNLP: System Demonstrations*, 2020.
[10] C. Wang *et al.*, "Robust text-to-sql generation with execution-guided decoding," *arXiv preprint arXiv:1807.03100*, 2018.
[11] T. Yu and contributors, "Spider: Official repository (evaluation scripts and data)," GitHub repository, 2018.
[12] ——, "test-suite-sql-eval: Semantic evaluation for text-to-sql," GitHub repository, 2018.
[13] OpenAI, "Gpt-4 technical report," Technical report, 2023.
[14] SQLAlchemy Project, "Sqlalchemy documentation," Official documentation, 2024.
[15] C. T. Hemphill, J. J. Godfrey, and G. R. Doddington, "The atis spoken language systems pilot corpus," *Proceedings of the DARPA Speech and Natural Language Workshop*, 1990.
[16] J. M. Zelle and R. J. Mooney, "Learning to parse database queries using inductive logic programming," in *Proceedings of AAAI*, 1996.
[17] Google, "Google colaboratory," Product documentation, 2024.
[18] NVIDIA, "Nvidia t4 tensor core gpu (specifications)," Product page / datasheet, 2024.
[19] PyTorch Contributors, "Pytorch documentation," Official documentation, 2024.
[20] H. Face, "Accelerate," GitHub repository, 2022.
[21] psycopg contributors, "psycopg2 documentation," Official documentation, 2024.
[22] andialbrecht and contributors, "sqlparse: A non-validating sql parser for python," GitHub repository, 2024.
[23] Docker, Inc., "Docker documentation," Official documentation, 2024.
[24] Python Software Foundation, "sqlite3 — db-api 2.0 interface for sqlite databases," Python documentation, 2024.